

プログラミング言語論

第12回 関数型プログラミング

篠埜 功

ML

- MLは関数型言語で、命令型の機能も持つ。(function-oriented imperative language)
- 関数型の特徴(Lispと同様): 関数が第一級の値(first class value)
 - 式の一部で関数が生成できる
 - 関数へ引数として関数を渡せる
 - 関数の返り値として関数を返せる
- MLの型システムは最もきれいで表現力が高いと考えられている
- Lisp/Algol系言語の重要な概念を含んでいる
- このスライドではStandard MLを用いて説明する。その他の関数型言語としてはOCaml、Haskellなどがある。

MLの型システム

- MLの型システムは安全(safe)
 - 型検査器が式がある型を持つと判定した場合、その式の評価が終わった時にその型の値を生成することが保証される。
 - (例)ある式が文字列へのポインタ型の場合、その式の値は文字列を保持しているメモリ領域を指すポインタであることが保証される。(解放された領域を指すポインタ(dangling pointer)や文字列以外の値を指すポインタにはならない。)

対話環境(interactive session)

```
- <式>;  
val it = <値の表示> : <型>
```

(例)

```
- 5+3-2;
```

```
val it = 6 : int
```

```
- it+3;
```

```
val it = 9 : int
```

```
- if true then 1 else 5;
```

```
val it = 1 : int
```

```
- 5=4;
```

```
val it = false : bool
```

MLの式の評価（実行）

- 式は構文解析、型検査、コンパイルの後実行される。
- 式が構文エラーや型エラーだった場合、コードは生成されず、実行もされない。
- （例）式 `if true then 3 else false` は構文エラーはないが、MLの型システムは `then part` と `else part` が同じ型を持つことを要求するので、型エラーとなる。
 - `if true then 3 else false;`

```
stdIn:5.1-5.26 Error: types of if branches do not agree [literal]
  then branch: int
  else branch: bool
in expression: if true then 3 else false
```

宣言

```
- val <id> = <exp>;  
val <id> = <値の表示> : <type>
```

valはvalueの頭文字3文字。宣言が入力されると、右辺の式が評価され、左辺の識別子に束縛される。

(例)

```
- val x = 7+2;  
val x = 9 : int  
- val y = x+3;  
val y = 12 : int  
- val z = x*y-(x div y);  
val z = 108 : int
```

(参考)

- MLでは/はreal型(浮動小数)の割り算に用いられる。
- Cの暗黙の型変換のように、int型からreal型への型変換が自動的には行われない。

(例)

- 9/12;

stdIn:1.2-1.6 Error: operator and operand
don't agree [literal]

operator domain: real * real

operand: int * int

in expression: 9 / 12

関数宣言

```
- fun <id> <arguments> = <exp>;  
val <id> = fn : <引数の型> -> <結果の型>
```

(例)

```
- fun f (x) = x+5;
```

```
val f = fn : int -> int
```

これは関数(int -> int型)をfに束縛する。

```
- val f = fn x => x+5;
```

```
val f = fn : int -> int
```

のように宣言することもできる。

fn x => x+5はラムダ式の $\lambda x. x + 5$ に対応する。

代入について

MLでは識別子の値は代入によって変更することができない。例えば、`val x = 3`と宣言されると`x`の値は常に3である。(MLの宣言はconstantを導入する。)MLで代入可能な変数を宣言するにはreference cellを用いる。

(例) - `val x = ref 3;`
 `val x = ref 3 : int ref`
 - `!x;`
 `val it = 3 : int`
 - `x:=4;`
 `val it = () : unit`
 - `!x;`
 `val it = 4 : int`

CやPascalにおける識別子

- CやPascalの識別子は、int型の変数は代入可能な変数であるが、関数宣言をした場合に関数名はconstantであり、代入によって別の関数に変更することはできない。つまりCやPascalでは型に応じてconstantかどうかが決まる。

型(type)

- 型は、値(value)の集合と操作(operation)の集合から成る。
- 型は、型式(type expression)を用いて表される。

(型式の
定義)

$\langle \text{type-exp} \rangle ::=$
 $\langle \text{type-name} \rangle$
 | $\langle \text{type-exp} \rangle \rightarrow \langle \text{type-exp} \rangle$
 | $\langle \text{type-exp} \rangle * \langle \text{type-exp} \rangle$
 | $\langle \text{type-exp} \rangle$ **list**
 | $\{ \langle \text{id} \rangle : \langle \text{type-exp} \rangle, \dots, \langle \text{id} \rangle : \langle \text{type-exp} \rangle \}$

基本型(basic type)

- 値がatomicな場合、その型は基本型であるという。値がそれ以上分解できない場合その値はatomicであるという。
 - (例) 集合{true, false}の値はatomicな値である。
- 基本型の値に対する操作は型毎に定義されており、等しさの比較などがある。
 - (例) $2=2$ はtrue, $2\neq 2$ はfalse。

MLの基本型

- unit型

- `() : unit`
- 戻り値の必要ない関数の戻り値、引数の必要ない関数の引数に用いられる。(Cで言えばvoid型)

- bool型

- `true: bool, false : bool`
- if式 `if e1 then e2 else e3` においてe1はbool型、e2とe3は何らかの同じ型を持たなければならない。else部分のないif式はない。
- 論理演算子 `andalso, orelse, not` はCの `&&, ||, !` に相当する。

MLの基本型(続き)

- int型
 - $0, 1, 2, \dots, -1, -2, \dots$: int
 - $+, -, *, \text{div} : \text{int} * \text{int} \rightarrow \text{int}$ (これらは中置演算子)
- string型
 - ダブルクォートで囲んだ記号列
 - 文字列連結演算子 $^ : \text{string} * \text{string} \rightarrow \text{string}$ (中値)
- real型
 - $1.0, 2.0, 3.14159, 4.4444, \dots$: real
 - $+, -, *$ は real, int どちらにも適用可。(ただし左右が同じ型でなければならない)

real関数でintからrealへの変換を行う。例えば`real(3)`は3.0。MLは型推論を行うので、明示的な変換を行うのは仕方がない。

組(tuple)、直積型(product type)

- 2つの型 A, B の直積型 $A * B$ は型 A の値と型 B の値の対から成る。
 - (例)(1, “one”)は整数1と文字列“one”から成る対である。
- n 個の型の直積 $A_1 * A_2 * \dots * A_n$ は組 $(a_1, a_2, \dots, a_n)$ から成る(a_i は型 A_i の値, $1 \leq i \leq n$)。
- 直積型に対する演算は、対の最初の要素と2番目の要素を取り出す関数であり以下のように定義される。

fun first (x,y) = x;

fun second (x,y) = y;

n 個の直積型に対する演算も同様に定義される。

組(tuple)の例

対(pair)、3つ組(triple)、4つ組(quadruple)、...

(組の例) - (3, 4);

```
val it = (3,4) : int * int
```

- (4, 5.5, true);

```
val it = (4,5.5,true) : int * real * bool
```

- ("Taro", "Jiro", 5);

```
val it = ("Taro","Jiro",5) : string * string * int
```

(組の要素
の取得例)

- #2 (3, 4);

```
val it = 4 : int
```

- #1 ("Taro", "Jiro", 5);

```
val it = "Taro" : string
```

レコード(record)

Cの構造体、Pascalのレコードに対応する。レコードは組と似ているが、各要素に名前がついているのが異なる。

(例) - {first_name="Donald", last_name="Knuth"};
val it =
{first_name="Donald",last_name="Knuth"}
: {first_name:string, last_name:string}

リスト(list)

- リストは有限の長さの要素の列である。
- 型 `A list` は `A` 型の要素のリスト全てから成る。
 - (例) `int list` は整数のリスト全てから成る。
- リストの要素は `[ と ]` の間にコンマで区切って書かれる。空リストは `[]` あるいは `nil` と書かれる。
 - (例) `[1,2,3]` は3つの整数 `1,2,3` のリストであり、`["red", "white", "blue"]` は3つの文字列 `"red", "white", "blue"` のリストである。
 - `cons` は `::` で記述する。 (例)

```
- 3::nil;  
val it = [3] : int list  
- 4::5::it;  
val it = [4,5,3] : int list
```

リスト(続き)

Lispにおけるリストとは異なり、MLのリストは要素がすべて同じ型でなければならない。

(例) - [1,2,3,4];
 val it = [1,2,3,4] : int list
 - [true,false];
 val it = [true,false] : bool list
 - ["red","blue"];
 val it = ["red","blue"] : string list
 - [fn x=>x+1, fn x=>x+2];
 val it = [fn,fn] : (int -> int) list

リストに対する操作

`null(x)` `x`が空リストなら`true`, そうでなければ`false`

`hd(x)` リスト`x`の最初の要素

`tl(x)` リスト`x`の最初の要素を除いたリスト

`a::x` 最初の要素が`a`で残りのリストが`x`のリスト

(例) `null []` は`true`

(例) `null [1,2,3]` は`false`

(例) `[1,2,3] = 1::[2,3] = 1::2::[3] = 1::2::3::[]`

`::`は右結合であり、`1::2::[3]`は`1::(2::[3])`と同じであり、`1::2::3::[]`は`1::(2::(3::[]))`と同じである。

パターン(pattern)

val宣言の左辺は識別子1つではなく、パターンを用いることができる。

```
val <pattern> = <exp>;
```

$\langle \text{pattern} \rangle ::= \langle \text{id} \rangle \mid \langle \text{tuple} \rangle \mid \langle \text{cons} \rangle \mid \langle \text{record} \rangle \mid \langle \text{constr} \rangle$

$\langle \text{tuple} \rangle ::= (\langle \text{pattern} \rangle, \dots, \langle \text{pattern} \rangle)$

$\langle \text{cons} \rangle ::= \langle \text{pattern} \rangle :: \langle \text{pattern} \rangle$

$\langle \text{record} \rangle ::= \{ \langle \text{id} \rangle = \langle \text{pattern} \rangle, \dots, \langle \text{id} \rangle = \langle \text{pattern} \rangle \}$

$\langle \text{constr} \rangle ::= \langle \text{id} \rangle (\langle \text{pattern} \rangle, \dots, \langle \text{pattern} \rangle)$

(注意) パターン中に同じ変数が2回以上現れてはいけない等の制約がある。(BNFでは表現できないので別途checkされる。)
nilは<constr>であり、引数のない構成子である。

パターンの例

```
- val t = (1,2,3);  
val t = (1,2,3) : int * int * int  
- val (x,y,z) = t;  
val x = 1 : int  
val y = 2 : int  
val z = 3 : int  
- val a::b = [1,2,3];  
val a = 1 : int  
val b = [2,3] : int list  
- val (a,b::c) = (1,[2,3]);  
val a = 1 : int  
val b = 2 : int  
val c = [3] : int list
```

パターンを用いた関数宣言

```
fun <id> <pattern> = <exp>
```

```
fun <id> <pattern> = <exp>  
  | ...  
  | <id> <pattern> = <exp>
```

のいずれかの形で宣言できる。

(例) - fun f (x,y) = x+y;
 val f = fn : int * int -> int
 - fun length nil = 0
 | length (x::xs) = 1 + length xs;
 val length = fn : 'a list -> int

(注意) 上記の例のfは2引数関数のように見えるが、対を1つ引数にとる1引数関数である。

(続き)

```
- length ["a","b","c"];  
val it = 3 : int
```

関数lengthの宣言は2つの節からなり、上から順にパターンマッチングが行われる。引数がnilにマッチするとき(引数が空リストのとき)はlengthは0を返し、そうでない時は $x::xs$ とパターンマッチングが行われる。

データ型を定義する場合はdatatype宣言を用いる。
(例) datatype tree = LEAF of int | NODE of (tree * tree)
の宣言下で以下のような関数が定義できる。

```
- fun inTree(x, LEAF y)=x=y  
  | inTree(x, NODE(y,z))=inTree(x,y) orelse inTree(x,z);  
val inTree = fn : int * tree -> bool
```